VECTOR

MICROSAR Diagnostic over IP for vehicle internal communication

Technical Reference

Version 2.0.0

Authors	Michael Dangelmaier, Jens Bauer
Status	Released

Document Information

History

Author	Date	Version	Remarks
Michael Dangelmaier	2019-07-18	1.0	Creation of document
Michael Dangelmaier	2019-12-18	2.0	Fixed some minor issues, Message structure for DoIP light, Configuration automation scripts, Overwrite Logical Address API

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_DefaultErrorTracer.pdf	V4.2.2
[2]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	V4.2.2
[3]	ISO	ISO 13400-2 2012	IS 2012



Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

1 Component History 6

2 Introduction 7
2.1 Architecture Overview 8

3 Functional Description 9
3.1 Features 9
3.2 Initialization 9
3.3 States 9
3.4 Main Functions 9
3.5 Error Handling 10
3.5.1 Development Error Reporting 10
3.5.2 Production Code Error Reporting 11

4 Integration 12
4.1 Embedded Implementation 12
4.2 Critical Sections 12
4.3 Main Functions 13

5 API Description 14
5.1 Type Definitions 14
5.2 Services provided by DoIPInt 14
5.2.1 DoIPInt_Init 14
5.2.2 DoIPInt_InitMemory 14
5.2.3 DoIPInt_MainFunction 15
5.2.4 DoIPInt_TpTransmit 15
5.2.5 DoIPInt_TpCancelTransmit 16
5.2.6 DoIPInt_TpCancelReceive 16
5.2.7 DoIPInt_GetVersionInfo 17
5.2.8 DoIPInt_OverwriteLogicalAddresses 17
5.3 Services used by DoIPInt 18
5.4 Callback Functions 18
5.4.1 DoIPInt_SoConModeChg 18
5.4.2 DoIPInt_SoAdTpCopyTxData 19
5.4.3 DoIPInt_SoAdTpTxConfirmation 20
5.4.4 DoIPInt_SoAdTpStartOfReception 20
5.4.5 DoIPInt_SoAdTpCopyRxData 21
5.4.6 DoIPInt_SoAdTpRxIndication 21

6 Configuration 23

© 2020 Vector Informatik GmbH

6.1 Configuration Variants 23
6.2 Configuration Design 23
6.3 Configuration Automation 24
6.4 Feature configuration 25
6.4.1 Configurable message structure 25

7 Glossary and Abbreviations 26
7.1 Glossary 26
7.2 Abbreviations 26

8 Contact 27

Illustrations

Figure 2-1 DoIPInt roles in an IP based vehicle network 7
Figure 2-2 DoIPInt in MICROSAR architecture 8
Figure 2-3 Interfaces to adjacent modules of the DoIPInt 8
Figure 6-1 Configuration design overview 23
Figure 6-2 DoIPInt automation scripts 24
Figure 6-3 Header structure DoIP ISO 13400 25
Figure 6-4 Header structure DoIP light 25

Tables

Table 1-1 Component history 6
Table 3-1 Supported Features 9
Table 3-2 Service IDs 10
Table 3-3 Errors reported to DET 10
Table 4-1 Implementation files 12
Table 5-1 DoIPInt_Init 14
Table 5-2 DoIPInt_InitMemory 15
Table 5-3 DoIPInt_MainFunction 15
Table 5-4 DoIPInt_TpTransmit 16
Table 5-5 DoIPInt_TpCancelTransmit 16
Table 5-6 DoIPInt_TpCancelReceive 17
Table 5-7 DoIPInt_GetVersionInfo 17
Table 5-8 DoIPInt_OverwriteLogicalAddresses 18
Table 5-9 Services used by the DoIPInt 18
Table 5-10 DoIPInt_SoConModeChg 19
Table 5-11 DoIPInt_SoAdTpCopyTxData 19
Table 5-12 DoIPInt_SoAdTpTxConfirmation 20
Table 5-13 DoIPInt_SoAdTpStartOfReception 21
Table 5-14 DoIPInt_SoAdTpCopyRxData 21
Table 5-15 DoIPInt_SoAdTpRxIndication 22
Table 7-1 Glossary 26
Table 7-2 Abbreviations 26

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Creation of component
2.00.00	Added extended functionality (transmission queue, reception queue), Message structure for DoIP light, Re-structuring of DoIPInt configuration data to enable calibration

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the MICROSAR BSW module DoIPInt.

Supported Configuration Variants:	pre-compile	
Vendor ID:	DOIPINT_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	DOIPINT_MODULE_ID	255 decimal (according to ref. [2])

The DoIPInt implements a transport protocol for diagnostic data over the Internet Protocol. It is derived from the DoIP module according to ISO Standard 13400 (see [3]) and adapted to fit the vehicle internal communication needs.

DoIP offers a lot of features which are not needed within a vehicle (e.g. routing activation of a tester). Therefore, DoIPInt implements only the generic header with the payload type “diagnostic message” according to [3]. All other messages (e.g. Generic DoIP header negative acknowledge) are not supported.

DoIP specifies that a tester always initiates the TCP connection (client) and an ECU waits for incoming TCP connections (server). DoIPInt implements both roles to make it usable in each node inside a vehicle (see Figure 2-1).

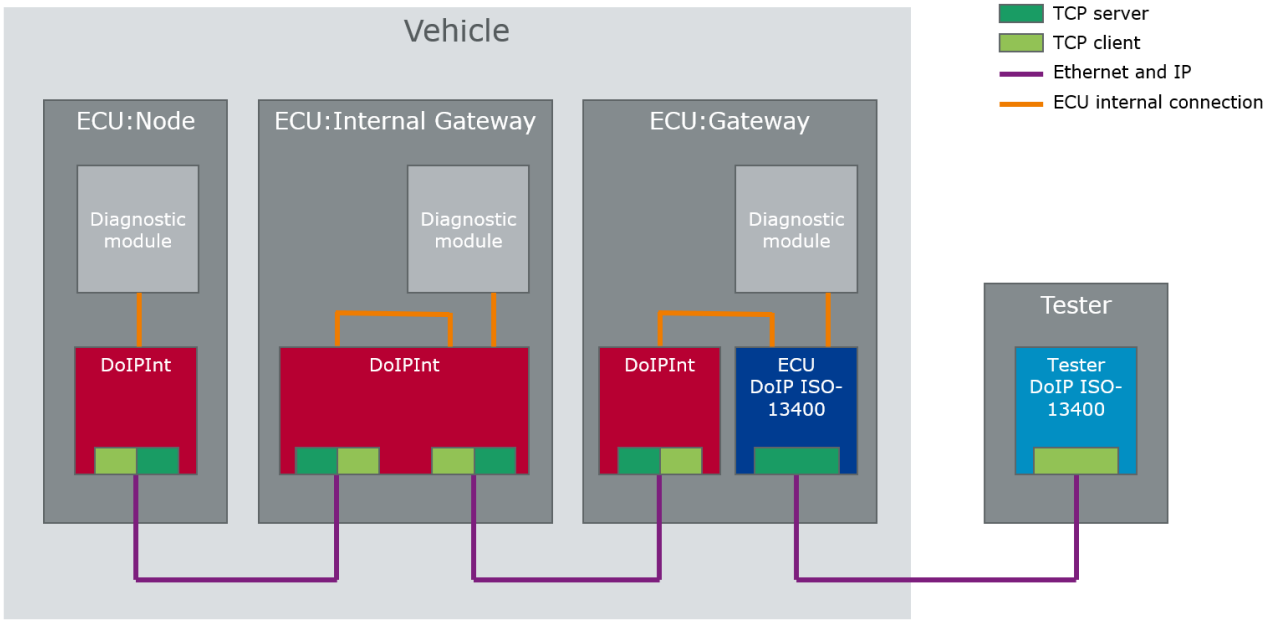


Figure 2-1 DoIPInt roles in an IP based vehicle network

2.1 Architecture Overview

The following figure shows where the DoIPInt is located in the MICROSAR architecture.

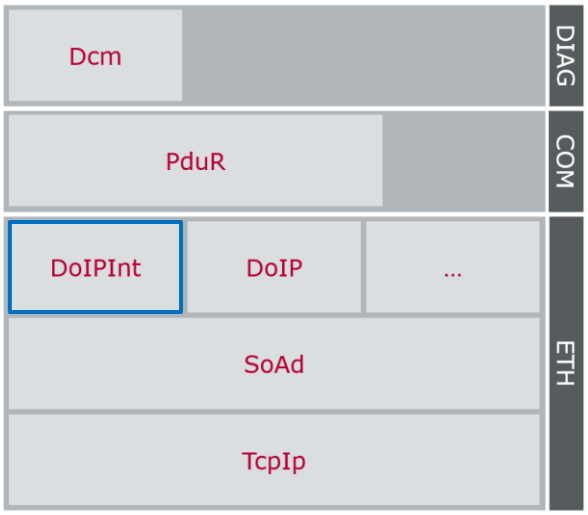


Figure 2-2 DoIPInt in MICROSAR architecture

The next figure shows the interfaces to adjacent modules of the DoIPInt. These interfaces are described in chapter 5.

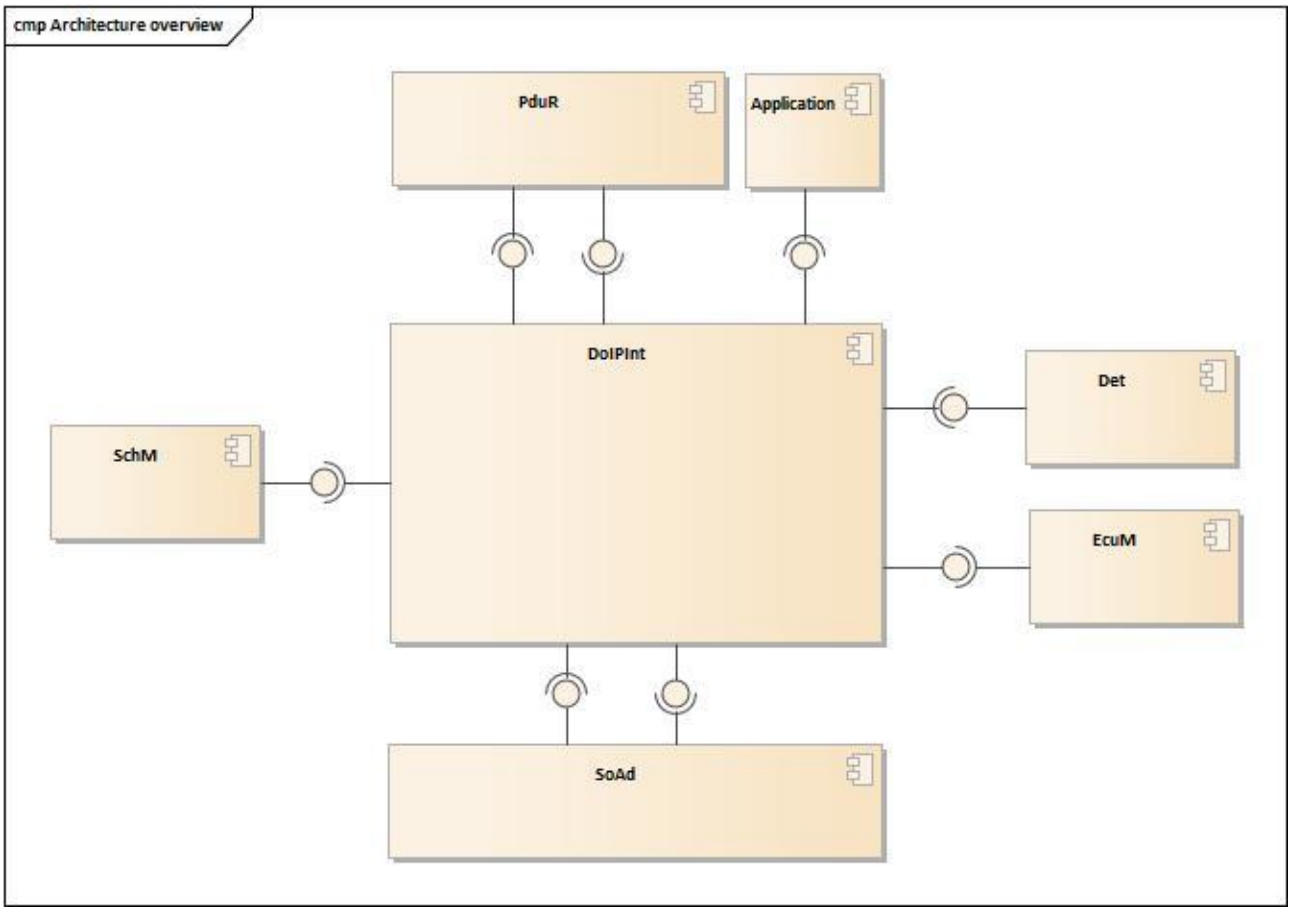


Figure 2-3 Interfaces to adjacent modules of the DoIPInt

3 Functional Description

3.1 Features

The features listed in the following tables cover the functionality specified for the DoIPInt.

The following features are supported:

Supported Features
Client TCP connection
Server TCP connection
Reopen of closed TCP connections
TCP Keep Alive mechanism
Transmission and reception of DoIP diagnostic messages over TCP connections
Cancellation of transmission and reception of DoIP diagnostic messages over TCP connections
Routing based on DoIP logical source and target address
Transmission queue for parallel transmission on same TCP connection
Reception retry on busy user
Configurable message structure
Overwritable logical addresses during runtime

Table 3-1 Supported Features

3.2 Initialization

The DoIPInt is initialized via a `DoIPInt_InitMemory()` call followed by a call to `DoIPInt_Init()`. A call to `DoIPInt_InitMemory()` reverts the initialization and a call to `DoIPInt_Init()` is required to initialize DoIPInt again.

3.3 States

DoIPInt provides an initialization state which is set by the services described in previous chapter.

3.4 Main Functions

The DoIPInt has one main function `DoIPInt_MainFunction()` which handles:

- The reopening of all closed TCP connections.
- The processing of all pending DoIPInt transmission requests.
- The processing of all pending DoIPInt reception retries.

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [1], if development error reporting is enabled (i.e. pre-compile parameter `DOIPINT_DEV_ERROR_REPORT ==STD_ON`).

If another module is used for development error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Det_ReportError()`.

The reported DoIPInt ID is 255.

The reported service IDs identify the services which are described in 5.2. The following table presents the service IDs and the related services:

Service ID	Service
0x00	DoIPInt_Init()
0x01	DoIPInt_MainFunction()
0x02	DoIPInt_TpTransmit()
0x03	DoIPInt_TpCancelTransmit()
0x04	DoIPInt_TpCancelReceive()
0x05	DoIPInt_GetVersionInfo()
0x06	DoIPInt_SoConModeChg()
0x07	DoIPInt_SoAdTpCopyTxData()
0x08	DoIPInt_SoAdTpTxConfirmation()
0x09	DoIPInt_SoAdTpStartOfReception()
0x0A	DoIPInt_SoAdTpCopyRxData()
0x0B	DoIPInt_SoAdTpRxIndication()
0x0C	DoIPInt_OverwriteLogicalAddresses()

Table 3-2 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
0x00	DOIPINT_E_NO_ERROR
0x01	DOIPINT_E_PARAM_CONFIG
0x02	DOIPINT_E_UNINIT
0x03	DOIPINT_E_PARAM
0x04	DOIPINT_E_PARAM_POINTER

Table 3-3 Errors reported to DET

3.5.2 Production Code Error Reporting

DoIPInt does not report production code related errors.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR DoIPInt into an application environment of an ECU.

4.1 Embedded Implementation

The delivery of the DoIPInt contains these source code files:

File Name	Description	Integration Task
DoIPInt.c	Static source file	-
DoIPInt.h	Static header file	-
DoIPInt_Cbk.h	Static header file for callback functions	-
DoIPInt_Connection.c	Static source file of sub-module which handles connections	-
DoIPInt_Connection.h	Static header file of sub-module which handles connections	-
DoIPInt_Hdr.c	Static source file of sub-module which handles the message header	-
DoIPInt_Hdr.h	Static header file of sub-module which handles the message header	-
DoIPInt_Priv.h	Static header file for DoIPInt internal usage	-
DoIPInt_Rx.c	Static source file of sub-module which handles reception	-
DoIPInt_Rx.h	Static header file of sub-module which handles reception	-
DoIPInt_Tx.c	Static source file of sub-module which handles transmission	-
DoIPInt_Tx.h	Static header file of sub-module which handles transmission	-
DoIPInt_Types.h	Static header file containing types	-
DoIPInt_Util.c	Static source file of sub-module which provides various utility	-
DoIPInt_Util.h	Static header file of sub-module which provides various utility	-
DoIPInt_Lcfg.c	Generated source file (e.g. RAM/ROM mapping tables)	-
DoIPInt_Lcfg.h	Generated header file	-
DoIPInt_Cfg.h	Generated header file for configuration parameter	-

Table 4-1 Implementation files

4.2 Critical Sections

All services and callbacks of DoIPInt may be called in interrupt or task context. Thus, a synchronization mechanism is implemented to guarantee data consistency.

The synchronization mechanism defined by AUTOSAR covers the entering and leaving of so called critical sections.

The implementation of the critical sections must avoid that multiple relevant tasks or interrupt service routines can enter each of the critical sections more than once at the same time.

Relevant interrupt services in the DoIPInt context are interrupt services that originate from physical bus events (Ethernet, CAN, LIN, FlexRay etc.).

Relevant tasks in the DoIPInt context are all tasks which call DoIPInt API functions. Usually these tasks are limited to tasks on which other BSW modules (TcpIp, SoAd, DoIP etc.) are mapped to.

A critical section can be handled by using the so called “Exclusive Areas”. The DoIPInt defines the following exclusive area:

> DOIPINT_EXCLUSIVE_AREA_0 is used whenever memory accesses must be protected from accesses of interrupting calls to services and callbacks of DoIPInt. This exclusive area may be entered in interrupt or task context. The frequency of entering and leaving this area will be very high. The average length of stay in the area is low.

For an implementation of the critical section it could be sufficient to:

> Disable all bus relevant interrupts of all buses related to calls to DoIPInt API functions (e.g. gateway use-case).

> Disable all Ethernet bus relevant interrupts if all modules calling DoIPInt API functions are mapped to one task (e.g. SchM task) or a non-preemptive OS is used.

Please note that these are only examples and that the actual implementation of the critical sections is highly dependent on the platform architecture and the system configuration.

4.3 Main Functions

DoIPInt expects that an own main function does not interrupt the main functions of related modules and is not interrupted by main functions of related modules.

The DoIPInt related modules are the SoAd and the TcpIp module.



Caution
Consider the mentioned main function expectations when using preemptive tasks (e.g. run all related modules in the same task to prevent interruption).

5 API Description

For an interfaces overview please see Figure 2-3.

5.1 Type Definitions

DoIPInt does not define user related types.

5.2 Services provided by DoIPInt

5.2.1 DoIPInt_Init

Prototype	
`void DoIPInt_Init (const DoIPInt_ConfigType *ConfigPtr)`	
Parameter	
ConfigPtr [in]	Configuration structure for initializing the module.
Return code	
void	none
Functional Description	
Initializes component. Service to initialize the module DoIPInt. It initializes all variables and sets the module state to initialized.	
Particularities and Limitations	
Interrupts are disabled and module main function is not triggered.	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-1 DoIPInt_Init

5.2.2 DoIPInt_InitMemory

Prototype	
`void DoIPInt_InitMemory (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Initializes *_INIT_*-variables. Service to initialize module global variables at power up. This function initializes the variables in *_INIT_* sections. Used in case they are not initialized by the startup code.	

Particularities and Limitations
Interrupts are disabled and module main function is not triggered.
Call context
> TASK > This function is Synchronous > This function is Non-Reentrant

Table 5-2 DoIPInt_InitMemory

5.2.3 DoIPInt_MainFunction

Prototype	
`void DoIPInt_MainFunction (void)`	
Parameter	
void	none
Return code	
void	none
Functional Description	
Schedules the DoIPInt (Entry point for scheduling) and handles connection states.	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-3 DoIPInt_MainFunction

5.2.4 DoIPInt_TpTransmit

Prototype	
`Std_ReturnType DoIPInt_TpTransmit (PduIdType DoIPIntPduRTxId, const PduInfoType *DoIPIntPduRTxInfoPtr)`	
Parameter	
DoIPIntPduRTxId [in]	DoIP unique identifier of the PDU to be transmitted by the PduR.
DoIPIntPduRTxInfoPtr [in]	PDU information structure which contains the length of the message.
Return code	
Std_ReturnType	E_OK Transmit request was accepted.
Std_ReturnType	E_NOT_OK Transmit request was not accepted.
Functional Description	
Requests transmission of a TP-PDU as diagnostic message. -	

Particularities and Limitations
-
Call context
> TASK|ISR2 > This function is Non-Reentrant

Table 5-4 DoIPInt_TpTransmit

5.2.5 DoIPInt_TpCancelTransmit

Prototype	
`Std_ReturnType` **`DoIPInt_TpCancelTransmit`** `(PduIdType DoIPIntPduRTxId)`	
Parameter	
DoIPIntPduRTxId [in]	DoIP unique identifier of the PDU to be transmitted by the PduR.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Requests transmission cancellation of a TP-PDU. -	
Particularities and Limitations	
-	
Call context	
> TASK|ISR2 > This function is Non-Reentrant	

Table 5-5 DoIPInt_TpCancelTransmit

5.2.6 DoIPInt_TpCancelReceive

Prototype	
`Std_ReturnType` **`DoIPInt_TpCancelReceive`** `(PduIdType DoIPIntPduRRxId)`	
Parameter	
DoIPIntPduRRxId [in]	DoIP unique identifier of the PDU to be received by the PduR.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.
Functional Description	
Requests reception cancellation of a TP-PDU. -	

Particularities and Limitations
-
Call context
> TASK|ISR2 > This function is Non-Reentrant

Table 5-6 DoIPInt_TpCancelReceive

5.2.7 DoIPInt_GetVersionInfo

Prototype	
`void DoIPInt_GetVersionInfo (Std_VersionInfoType *Versioninfo)`	
Parameter	
Versioninfo [out]	Pointer to where to store the version information.
Return code	
void	none
Functional Description	
Returns the version information. DoIPInt_GetVersionInfo() returns version information, vendor ID and module ID of the component.	
Particularities and Limitations	
- Configuration Variant(s): DOIPINT_VERSION_INFO_API	
Call context	
> TASK|ISR2 > This function is Synchronous > This function is Reentrant	

Table 5-7 DoIPInt_GetVersionInfo

5.2.8 DoIPInt_OverwriteLogicalAddresses

Prototype	
`Std_ReturnType DoIPInt_OverwriteLogicalAddresses (DoIPInt_ChannelIdType ChannelId, uint16 LogicalLocalAddress, uint16 LogicalRemoteAddress)`	
Parameter	
ChannelId [in]	DoIPInt unique identifier of the channel.
LogicalLocalAddress [in]	Local address of channel.
LogicalRemoteAddress [in]	Remote address of channel.
Return code	
Std_ReturnType	E_OK Request was accepted.
Std_ReturnType	E_NOT_OK Request was not accepted.

Functional Description
Overwrites logical addresses until reinitialization.
Particularities and Limitations
-
Call context
> TASK|ISR2 > This function is Synchronous > This function is Non-Reentrant

Table 5-8 DoIPInt_OverwriteLogicalAddresses

5.3 Services used by DoIPInt

In the following table services provided by other components, which are used by the DoIPInt are listed. For details about prototype and functionality refer to the documentation of the providing component.

Component	API
DET	Det_ReportError
IpBase	IpBase_GetUint8
IpBase	IpBase_GetUint16
IpBase	IpBase_GetUint32
PduR	PduR_DoIPIntTpCopyTxData
PduR	PduR_DoIPIntTpTxConfirmation
PduR	PduR_DoIPIntTpStartOfReception
PduR	PduR_DoIPIntTpCopyRxData
PduR	PduR_DoIPIntTpRxIndication
SoAd	SoAd_OpenSoCon
SoAd	SoAd_CloseSoCon
SoAd	SoAd_TpTransmit
SoAd	SoAd_TpCancelTransmit
SoAd	SoAd_TpCancelReceive
VStdLib	VStdMemCpy

Table 5-9 Services used by the DoIPInt

5.4 Callback Functions

This chapter describes the callback functions that are implemented by the DoIPInt and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `DoIPInt_Cbk.h` by the DoIPInt.

5.4.1 DoIPInt_SoConModeChg

Prototype
`void DoIPInt_SoConModeChg (SoAd_SoConIdType SoConId, SoAd_SoConModeType Mode)`

Parameter	
SoConId [in]	Socket connection index specifying the socket connection with mode change.
Mode [in]	New socket connection mode.
Return code	
void	none
Functional Description	
Receives notification for socket connection mode change. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Synchronous > This function is Non-Reentrant	

Table 5-10 DoIPInt_SoConModeChg

5.4.2 DoIPInt_SoAdTpCopyTxData

Prototype
`BufReq_ReturnType` **`DoIPInt_SoAdTpCopyTxData`** `(PduIdType DoIPIntSoAdTxPduId, PduInfoType *DoIPIntSoAdTxInfoPtr, RetryInfoType *RetryPtr, PduLengthType *AvailableDataPtr)`

Parameter	
DoIPIntSoAdTxPduId [in]	DoIPInt unique identifier of the PDU transmitted by the SoAd.
DoIPIntSoAdTxInfoPtr [in]	Provides the destination buffer and the number of bytes to be copied.
RetryPtr [in]	Parameter is not supported (NULL_PTR).
AvailableDataPtr [out]	Indicates the remaining number of bytes that are available to be copied.
Return code	
BufReq_ReturnType	BUFREQ_OK Data has been copied to the transmit buffer completely as requested.
BufReq_ReturnType	BUFREQ_E_NOT_OK Data has not been copied. Request failed.
Functional Description	
Copies data for transmission to provided buffer. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Non-Reentrant	

Table 5-11 DoIPInt_SoAdTpCopyTxData

5.4.3 DoIPInt_SoAdTpTxConfirmation

Prototype	
`void DoIPInt_SoAdTpTxConfirmation (PduIdType DoIPIntSoAdTxPduId, Std_ReturnType Result)`	
Parameter	
DoIPIntSoAdTxPduId [in]	DoIPInt unique identifier of the PDU transmitted by the SoAd.
Result [in]	Result of the transmission.
Return code	
void	none
Functional Description	
Receives confirmation for transmitted PDU. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Non-Reentrant	

Table 5-12 DoIPInt_SoAdTpTxConfirmation

5.4.4 DoIPInt_SoAdTpStartOfReception

Prototype	
`BufReq_ReturnType DoIPInt_SoAdTpStartOfReception (PduIdType DoIPIntSoAdRxPduId, PduInfoType *DoIPIntSoAdRxInfoPtr, PduLengthType TpSduLength, PduLengthType *AvailableDataPtr)`	
Parameter	
DoIPIntSoAdRxPduId [in]	DoIPInt unique identifier of the PDU to be received from the SoAd.
DoIPIntSoAdRxInfoPtr [in]	Parameter is not supported (NULL_PTR).
TpSduLength [in]	Length of the PDU.
AvailableDataPtr [out]	Indicates the remaining number of bytes that are available to be copied.
Return code	
BufReq_ReturnType	BUFREQ_OK Reception request was accepted.
BufReq_ReturnType	BUFREQ_E_NOT_OK Reception request was not accepted.
Functional Description	
Indicates start of reception of a PDU. DoIPInt extracts the diagnostic messages from TCP stream. Therefore, function expects to be called with TpSduLength set to 0.	
Particularities and Limitations	
-	
Call context	

> TASK > This function is Non-Reentrant

Table 5-13 DoIPInt_SoAdTpStartOfReception

5.4.5 DoIPInt_SoAdTpCopyRxData

Prototype	
`BufReq_ReturnType` **`DoIPInt_SoAdTpCopyRxData`** `(PduIdType DoIPIntSoAdRxPduId, PduInfoType *DoIPIntSoAdRxInfoPtr, PduLengthType *AvailableDataPtr)`	
Parameter	
DoIPIntSoAdRxPduId [in]	DoIPInt unique identifier of the PDU to be received from the SoAd.
DoIPIntSoAdRxInfoPtr [in]	Provides the buffer and the number of bytes to be copied.
AvailableDataPtr [out]	Indicates the remaining number of bytes that are available to be copied.
Return code	
BufReq_ReturnType	BUFREQ_OK Data has been copied completely as requested.
BufReq_ReturnType	BUFREQ_E_NOT_OK Data has not been copied. Request failed.
Functional Description	
Copies data for reception from provided buffer. -	
Particularities and Limitations	
-	
Call context	
> TASK > This function is Non-Reentrant	

Table 5-14 DoIPInt_SoAdTpCopyRxData

5.4.6 DoIPInt_SoAdTpRxIndication

Prototype	
`void` **`DoIPInt_SoAdTpRxIndication`** `(PduIdType DoIPIntSoAdRxPduId, Std_ReturnType Result)`	
Parameter	
DoIPIntSoAdRxPduId [in]	DoIPInt unique identifier of the PDU to be received from the SoAd.
Result [in]	Result of the reception.
Return code	
void	none
Functional Description	
Receives indication for received PDU. Since DoIPInt extracts the diagnostic messages from TCP stream this indication also indicates that socket connection is closed.	

Particularities and Limitations
-
Call context
> TASK > This function is Non-Reentrant

Table 5-15 DoIPInt_SoAdTpRxIndication

6 Configuration

6.1 Configuration Variants

The DoIPInt supports the configuration variants

> VARIANT-PRE-COMPILE

6.2 Configuration Design

Figure 6-1 provides an overview over the configuration design.

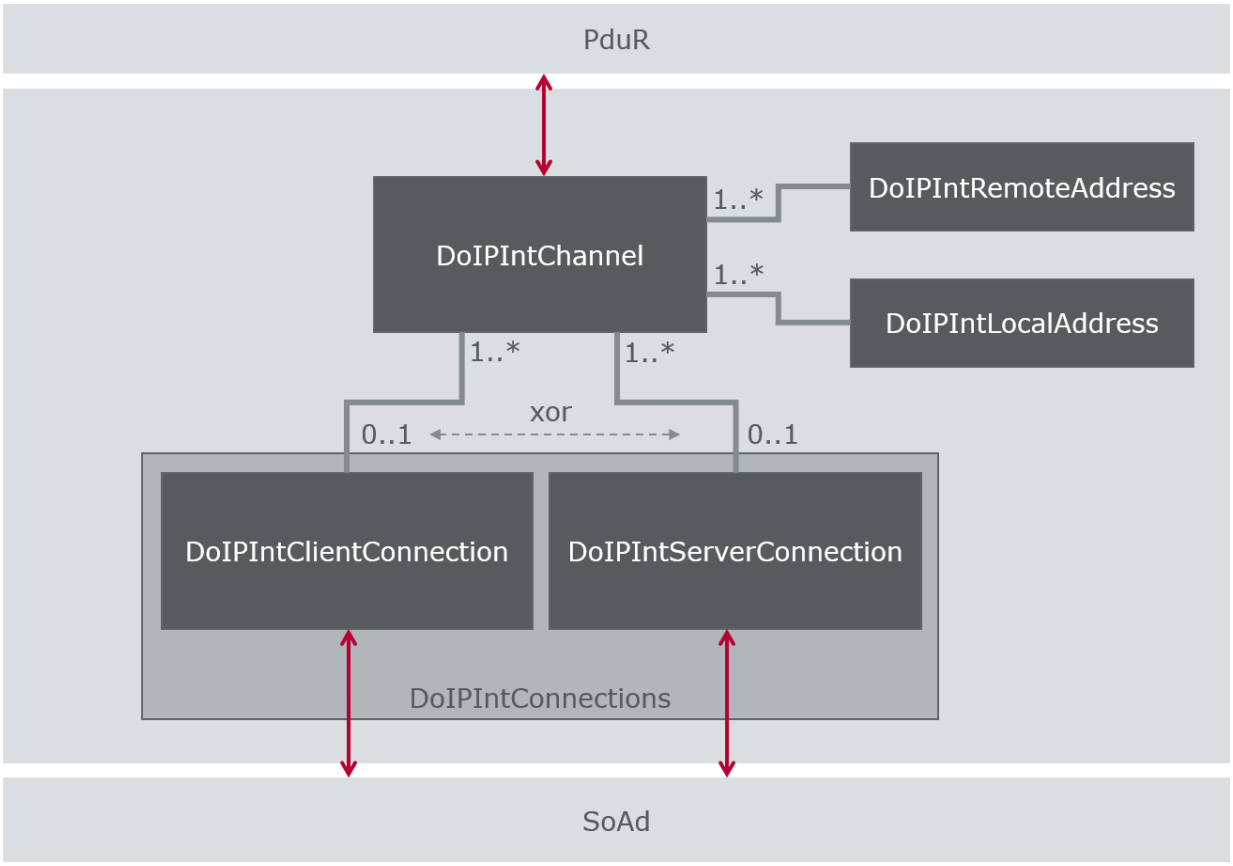


Figure 6-1 Configuration design overview

The DoIPIntConnections are the connections to the SoAd. Each DoIPIntClientConnection or DoIPIntServerConnection represents a separate TCP connection. Over these connections DoIPInt transmits or receives diagnostic messages. In case of DoIPIntClientConnection DoIPInt initiates the TCP connection. In case of DoIPIntServerConnection DoIPInt waits for an incoming TCP connection. It is possible to configure only DoIPIntClientConnection or DoIPIntServerConnection or both.

A DoIPIntChannel is the connection to the PduR. Over a DoIPIntChannel DoIPInt transmits or receives the user data of a diagnostic message (i.e. no header information). On transmission DoIPInt appends a DoIP header before the user data considering the logical

addresses of DoIPIntRemoteAddress and DoIPIntLocalAddress of the specific DoIPIntChannel. This diagnostic message is sent over the referenced DoIPIntClientConnection or DoIPIntServerConnection. On reception of a diagnostic message all DoIPIntChannels related to the DoIPIntClientConnection or DoIPIntServerConnection are searched to find the matching DoIPIntChannel. This is done by comparing the configured tuple of DoIPIntRemoteAddress and DoIPIntLocalAddress to the logical source and target address received in the DoIP header.

A DoIPIntRemoteAddress represents the logical address used in a diagnostic message as source address on reception and as target address on transmission.

A DoIPIntLocalAddress represents the logical address used in a diagnostic message as target address on reception and as source address on transmission.

For detailed configuration parameter descriptions please see the DoIPInt_bswmd.arxml file.

6.3 Configuration Automation

Enable the Script Task View within the DaVinci Configurator to see the automation scripts.

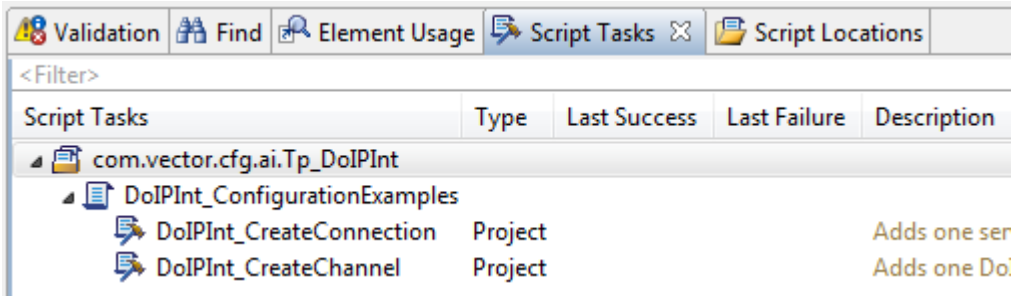


Figure 6-2 DoIPInt automation scripts

The DoIPInt provides two automation scripts which helps to easily create DoIPIntChannel, DoIPIntClientConnection and DoIPIntServerConnection.

The script task DoIPInt_CreateConnection adds one server/client connection for usage with DoIPInt to the configuration. This task configures DoIPInt, SoAd and EcuC for one DoIPInt connection. The connections/containers are created with a numerical postfix requested from the user. After the successful execution of the script task, manually assign the channels to the created connection in DoIPInt/DoIPIntConfigSet/DoIPIntChannel.

The script task DoIP_CreateChannel adds one DoIPInt channel to the configuration. Channel names/containers are postfixed with remote and/or local address. This task configures DoIPInt, PduR and EcuC for one DoIPInt channel. After the successful execution of the script task, manually assign the created channel to a connection in DoIPInt/DoIPIntConfigSet/DoIPIntChannel and complete the PduR routing path configuration.

6.4 Feature configuration

6.4.1 Configurable message structure

DoIPInt supports the header structure according to [3] (Figure 6-3) and an optimized “light” header structure (Figure 6-4). The structure is configurable globally (i.e. at runtime only one structure can be supported in parallel).

The configuration parameter `DoIPIntHeaderStructure` is used to select the required structure.

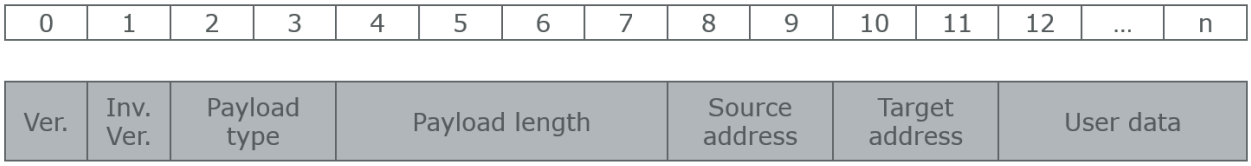


Figure 6-3 Header structure DoIP ISO 13400

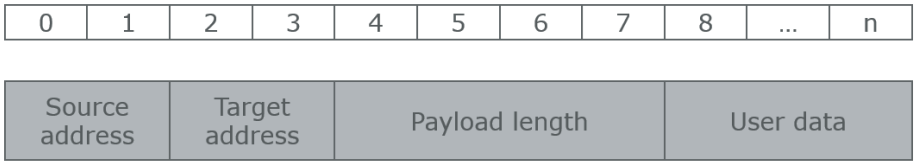


Figure 6-4 Header structure DoIP light

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
PduR	AUTOSAR module PDU Router
SchM	AUTOSAR module Schedule Manager
SoAd	AUTOSAR module Socket Adaptor
TcpIp	AUTOSAR module TcpIp

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
DET	Development Error Tracer
ECU	Electronic Control Unit
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
OS	Operating System
TCP	Transmission Control Protocol

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector.com